

MBA EM **ENGENHARIA
DE SOFTWARE**

**Arquitetura Limpa (Clean
Architecture) I-II**

Prof. Helder Prado Santos

A responsabilidade pela idoneidade, originalidade e licitude dos conteúdos didáticos apresentados é do professor.

Proibida a reprodução, total ou parcial, sem autorização.

Lei nº 9610/98

O Problema do Acoplamento

Sintomas de uma Arquitetura Ruim

- **Rigidez:** O sistema é difícil de mudar porque cada mudança afeta muitas outras partes.
- **Fragilidade:** Mudanças quebram partes do sistema que não têm relação conceitual com a parte alterada.
- **Imobilidade:** É difícil reutilizar o sistema em outros projetos porque ele envolve muitos riscos e esforço.

Objetivo: Transformar o "Big Ball of Mud" em um sistema estruturado.

Nobody:
My code:



| O Que é Arquitetura Limpa?

Proposta por **Robert C. Martin (Uncle Bob)** em 2012, não é uma "regra nova", mas uma consolidação de boas práticas (Hexagonal, Onion, Screaming Architecture).



Independência

O sistema não depende de bibliotecas ou frameworks. Eles são ferramentas, não o núcleo.



Testabilidade

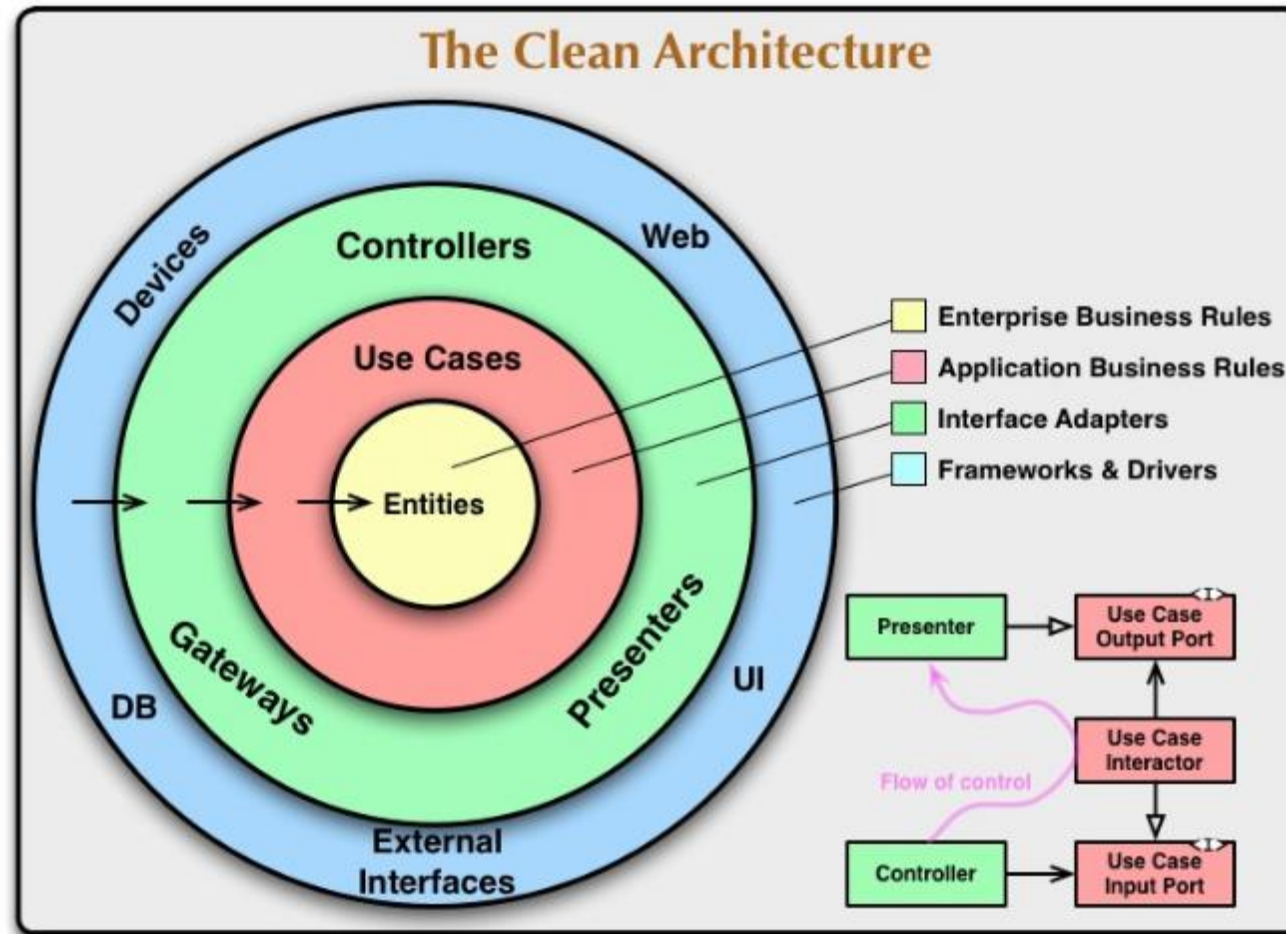
Regras de negócio podem ser testadas sem UI, BD, Servidor Web ou qualquer elemento externo.



Independência de BD

Você pode trocar Oracle por Mongo, ou SQL Server por CouchDB. As regras de negócio não sabem o que é um BD.

O Famoso Diagrama



| A Regra de Ouro: Dependência

"Dependências de código fonte devem apontar apenas para dentro."

- Nada num círculo interno pode saber algo sobre um círculo externo.
- Nomes de classes, funções ou variáveis declaradas externamente não devem ser mencionados internamente.
- O nível mais alto (Regras) não depende do nível mais baixo (Detalhes).

Violação Comum:

Importar uma biblioteca SQL dentro da sua Entidade de Domínio.

| 1. Entidades (Domínio)

Regras de Negócio Corporativas

São os objetos de negócio da aplicação. Elas encapsulam as regras mais gerais e de alto nível.

- Se é uma aplicação corporativa, as entidades podem ser usadas por diferentes aplicações da empresa.
- Se é uma aplicação única, são os objetos do domínio principal.
- **Não mudam** quando a página muda, quando a segurança muda ou quando o banco de dados muda.



Entidades: Código Prático

```
from uuid import UUID
from domain.task.task_entity import Task
from typing import List

class User:

    id: UUID
    name: str
    tasks: List[Task]

    def __init__(self, id: UUID, name: str):
        self.id = id
        self.name = name
        self.tasks = []
        self.validate()

    def validate(self):
        if not isinstance(self.id, UUID):
            raise Exception("id must be an UUID")

        if not isinstance(self.name, str) or len(self.name) == 0:
            raise Exception("name is required")

    def collect_tasks(self, tasks: List[Task]) -> None:
        self.tasks.extend(tasks)

    def count_pending_tasks(self) -> int:
        count = 0
        for task in self.tasks:
            if task.completed is not True:
                count += 1
```

Observações:

- Possuem identidade própria
- Concentram regras e comportamentos de negócio
- São independentes de infraestrutura
- Devem proteger seu estado interno

| 2. Casos de Uso (Application)

Regras de Negócio da Aplicação

Contêm regras específicas da aplicação. Orquestram o fluxo de dados para e das entidades.

- Direcionam as entidades para usar suas regras de negócio corporativas.
- Não afetam as entidades.
- Não são afetados por UI, BD ou Frameworks.
- Exemplos: *Criar Pedido, Matricular Aluno, Gerar Extrato.*



Casos de Uso: Inversão de Dependência

Antes de implementar o caso de uso, definimos a interface do repositório. O Caso de uso **não sabe** como salvar no banco, ele apenas sabe que **precisa** salvar.

```
import { User } from '../domain/entities/User';

// Contrato que a camada de Infra deve cumprir
export interface UserRepositoryInterface {
  save(user: User): Promise<void>;
  findByEmail(email: string): Promise<User | null>;
}
```

Isso garante que o Caso de Uso dependa de uma abstração, não de uma implementação (Princípio "D" do SOLID).

Casos de Uso: Implementação

```
from domain.__seedwork.use_case_interface import UseCaseInterface
from domain.user.user_repository_interface import
UserRepositoryInterface
from usecases.user.add_user.add_user_dto import (
    AddUserInputDto,
    AddUserOutputDto
)
from domain.user.user_entity import User
import uuid

class AddUserUseCase(UseCaseInterface):

    def __init__(self, user_repository: UserRepositoryInterface):
        self.user_repository = user_repository

    def execute(self, input: AddUserInputDto) -> AddUserOutputDto:

        user = User(
            id=uuid.uuid4(),
            name=input.name
        )

        self.user_repository.add_user(user=user)

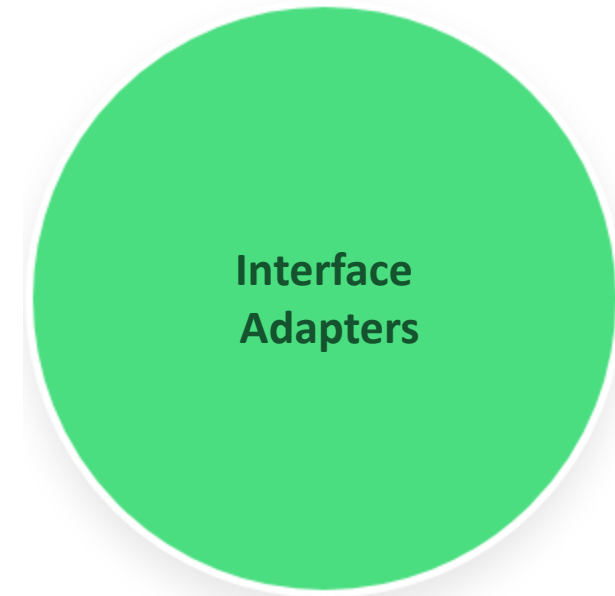
        return AddUserOutputDto(
            id=user.id,
            name=user.name
```

| 3. Adaptadores de Interface

Conversores de Dados

O objetivo é converter dados no formato mais conveniente para os casos de uso e entidades, para o formato mais conveniente para agentes externos (Web, BD).

- **Controllers:** Recebem input (HTTP Request) e convertem para DTO do Caso de Uso.
- **Presenters:** Recebem o retorno do Caso de Uso e formatam para a View (HTML, JSON).
- **Gateways:** Implementações de interfaces para serviços externos.



Adaptadores: Web Controller

```
export class UserController {
  constructor(private useCase: RegisterUser) {}

  // Recebe objetos de framework (Request, Response)
  async handle(request: Request, response: Response) {
    try {
      const { name, email } = request.body;

      // Chama o caso de uso (Agnóstico a HTTP)
      const result = await this.useCase.execute({
        id: UUID.gen(),
        name,
        email,
      });

      return response.status(201).json(result);
    } catch (e) {
      return response.status(400).json({
        error: e.message,
      });
    }
  }
}
```

Responsabilidades:

- Desacoplar o HTTP do Domínio.
- Extrair dados da requisição.
- Invocar o Caso de Uso.
- Devolver a resposta correta.

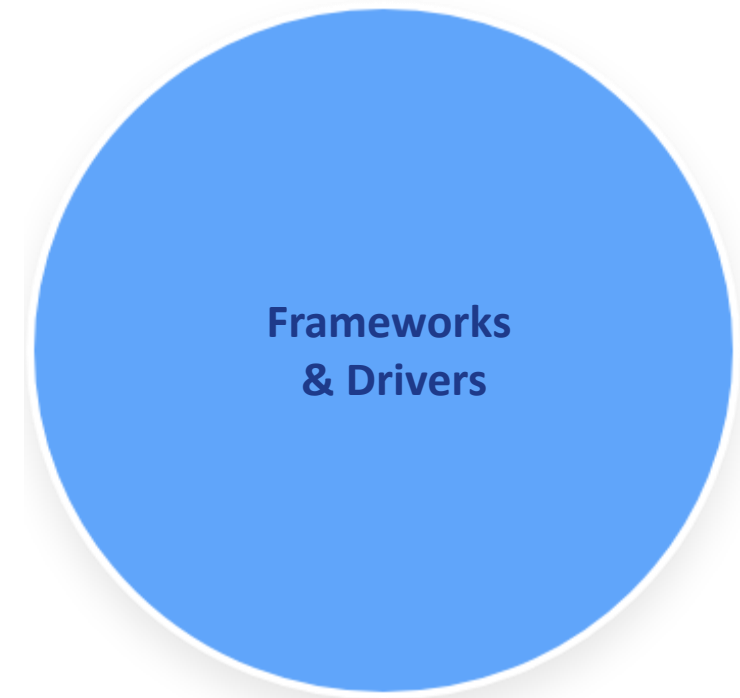
| 4. Frameworks & Drivers (Infra)

Onde vivem os detalhes

A camada mais externa. Geralmente composta por frameworks e ferramentas como:

- Banco de Dados (Postgres, Mongo).
- Web Frameworks (Express, NestJS, Spring).
- Dispositivos (UI, Impressoras).

Não escrevemos muito código aqui, além do código de "cola" que comunica com os círculos internos.



Infra: Implementação de Repositório

Aqui implementamos a interface definida na camada de Aplicação.

```
import { UserRepositoryInterface } from '../../../application/repositories';
import { User } from '../../../domain/entities/User';
import { UserModel } from './mongoose/UserSchema';

// Implementação concreta usando MongoDB (lib externa)
export class MongoUserRepository implements UserRepositoryInterface {
  async save(user: User): Promise<void> {
    // Converte Entidade de Domínio → Modelo do Mongo
    await UserModel.create({
      _id: user.id,
      name: user.name,
      email: user.email,
    });
  }

  async findByEmail(email: string): Promise<User | null> {
    const data = await UserModel.findOne({ email });

    if (!data) {
      return null;
    }

    // Reconstrói a Entidade de Domínio
    return new User(
      data._id.toString(),
      data.name,
      data.email
    );
  }
}
```

Cruzando Limites (Boundaries)

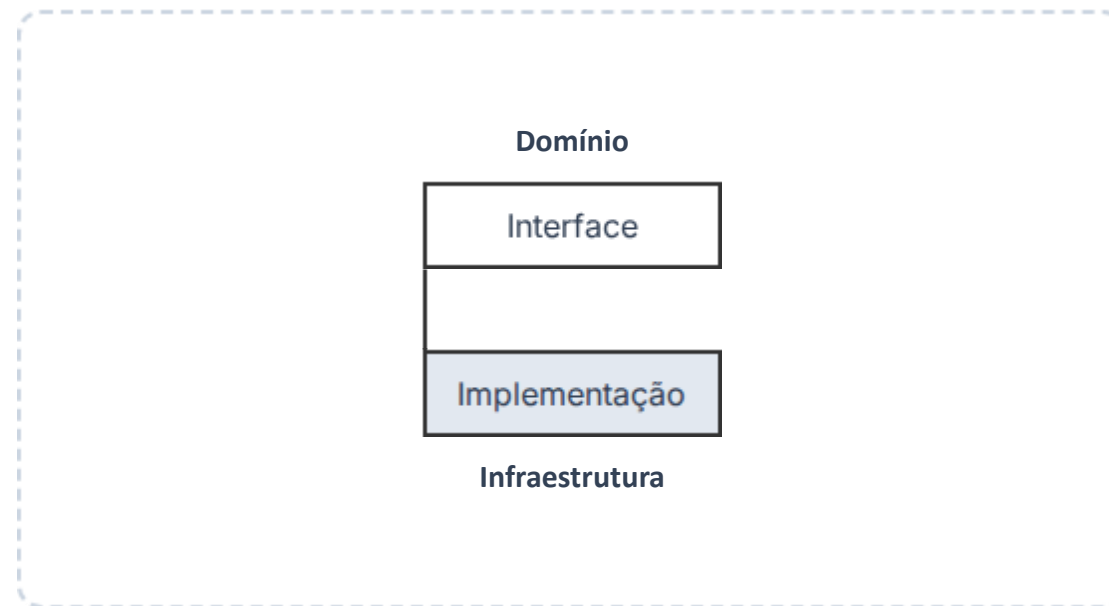
Fluxo de Controle vs. Dependência

O fluxo de controle (runtime) pode ir em qualquer direção (Controller chama UseCase, UseCase chama Repositório).

Porém, a **dependência de código fonte** deve sempre apontar contra o fluxo de controle quando cruzamos limites para fora.

Como? Polimorfismo (Interfaces).

O Use Case chama uma interface RepositoryInterface (que está na camada Domain), mas a implementação MongoRepository está na camada Infra.



| Estratégia de Testes



Testes Unitários

Testam Entidades e Casos de Uso. São rápidos (ms). Não usam Mocks complexos, apenas dados puros.



Testes de Integração

Testam a comunicação entre Adaptadores e Casos de Uso. Verificam se o Controller passa o DTO correto.



Testes E2E

Testam a Infraestrutura. Levantam o banco de dados real e fazem requisições HTTP reais.

Estrutura de Pastas Sugerida

```

└─ domain
  └─ __seedwork
  └─ task
  └─ user
    └─ __pycache__
    └─ user_entity.py
    └─ user_repository_interface.py
  └─ infrastructure
    └─ api
    └─ task
    └─ user / sqlalchemy
      └─ __pycache__
      └─ __init__.py
      └─ user_model.py
      └─ user_repository.py
  └─ usecases
    └─ task
    └─ user
      └─ add_user
        └─ __pycache__
        └─ __init__.py
        └─ add_user_dto.py
        └─ add_user_usecase.py
    └─ find_user
```

Screaming Architecture

A estrutura de pastas deve "gritar" sobre o que o sistema faz, não qual framework ele usa.

Ao abrir o projeto, você deve ver "Usuarios", "Pedidos", "Produtos", e não apenas "Models", "Views", "Controllers".

| Trade-offs (Nem tudo são flores)

Vantagens

- Desacoplamento total.
- Testabilidade extrema.
- Fácil manutenção a longo prazo.
- Independência de Frameworks.

Desvantagens

- Curva de aprendizado íngreme.
- Muito "boilerplate" (Muitos arquivos).
- Overhead mental para CRUDs simples.
- Duplicação de código (DTO vs Entity).

| Quando usar Arquitetura Limpa?

USE QUANDO:

- Projetos médios ou grandes, com regras de negócio complexas.
- Sistemas com longa vida útil, que precisarão evoluir ao longo do tempo.
- Times com vários desenvolvedores, facilitando organização e manutenção.
- Necessidade de testes automatizados, especialmente testes de negócio.

EVITE QUANDO:

- Projetos pequenos ou MVPs, onde simplicidade e velocidade são prioridade.
- Aplicações muito simples (CRUD básico) sem regras complexas.
- Prazos muito curtos, onde a sobrecarga arquitetural atrapalha.
- Times sem maturidade técnica, podendo gerar confusão e overengineering.

| Recapitulação

- **Entidades:** O núcleo do domínio, contendo as regras de negócio mais estáveis.
- **Casos de Uso:** Orquestram as regras de negócio e definem o fluxo da aplicação.
- **Adaptadores:** Convertem dados entre o mundo externo e o domínio interno.
- **Infraestrutura:** Implementações técnicas e ferramentas substituíveis.
- **Regras de Dependência:** As dependências sempre apontam para o núcleo da aplicação.

"Uma boa arquitetura permite adiar decisões críticas."



Sugestões de Leituras/Referências

MARTIN, Robert C. Arquitetura limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2018.

MARTIN, Robert C. Código limpo: habilidades práticas do Agile Software. Rio de Janeiro: Alta Books, 2009.

EVANS, Eric. Domain-Driven Design: atacando as complexidades no coração do software. Rio de Janeiro: Alta Books, 2016.

MARTIN, Robert C. (Uncle Bob). The Clean Code Blog. Disponível em: <https://blog.cleancoder.com>

Obrigado!

Helder Prado Santos | [linkedin.com/in/helderprado](https://www.linkedin.com/in/helderprado)

MBAUSP
ESALQ